Voice Scribe v3
Desktop Application

Johnraymond Yanez

Voice Scribe

April 5, 2026

Overview

- **Context** — v1 and v2 in brief
- **The Platform Ceiling** — what the browser cannot close
- **v3 Architecture** — Tauri, Python sidecar, platform layer
- **Roadmap** — four sub-milestones (v3.0–v3.3)
- **Key Design Decisions** — the choices and why
- **Prerequisites & Deferrals**

v1 — The Primitive (2026-04-02)

Goal: local speech-to-text that never leaves the machine.

What shipped:

- Record one utterance in the browser; POST to a local container
- ffmpeg transcodes; faster-whisper (large-v3, GPU) infers
- Transcript returned to an editable textarea; one-click copy
- CPU fallback (`small` model) validated

What it proved: a two-service localhost architecture — React on the host, FastAPI in Docker — can deliver accurate, private transcription with no cloud dependency and no round-trip latency beyond the GPU itself.

Intentionally out of scope: streaming, continuous recording, hotkeys, voice commands, server-side persistence. v1 was a primitive, not a product.

v2 — The Interaction Model (2026-04-04)

Built on v1 across four sequenced milestones:

Milestone	Key deliverable
v2.0	Hotkey system (`Ctrl+Shift+*`); settings panel with live model swap (threading-safe, VRAM cleanup)
v2.1	WebSocket streaming default; words appear as segments complete; HTTP POST fallback automatic
v2.2	Continuous recording; client-side VAD (Silero v4/WASM); paragraph breaks; drain-then-done
v2.3.x	Voice commands: `copy` and `clear` as single-word exact-segment matches; paragraph break timing fixed

What it proved: streaming, browser-side ML inference, and voice-driven control all work reliably at the browser layer. The interaction model is validated.

What it could not close: covered next.

What the Browser Cannot Do

v2 delivers a capable continuous dictation tool. One constraint cannot be engineered around:

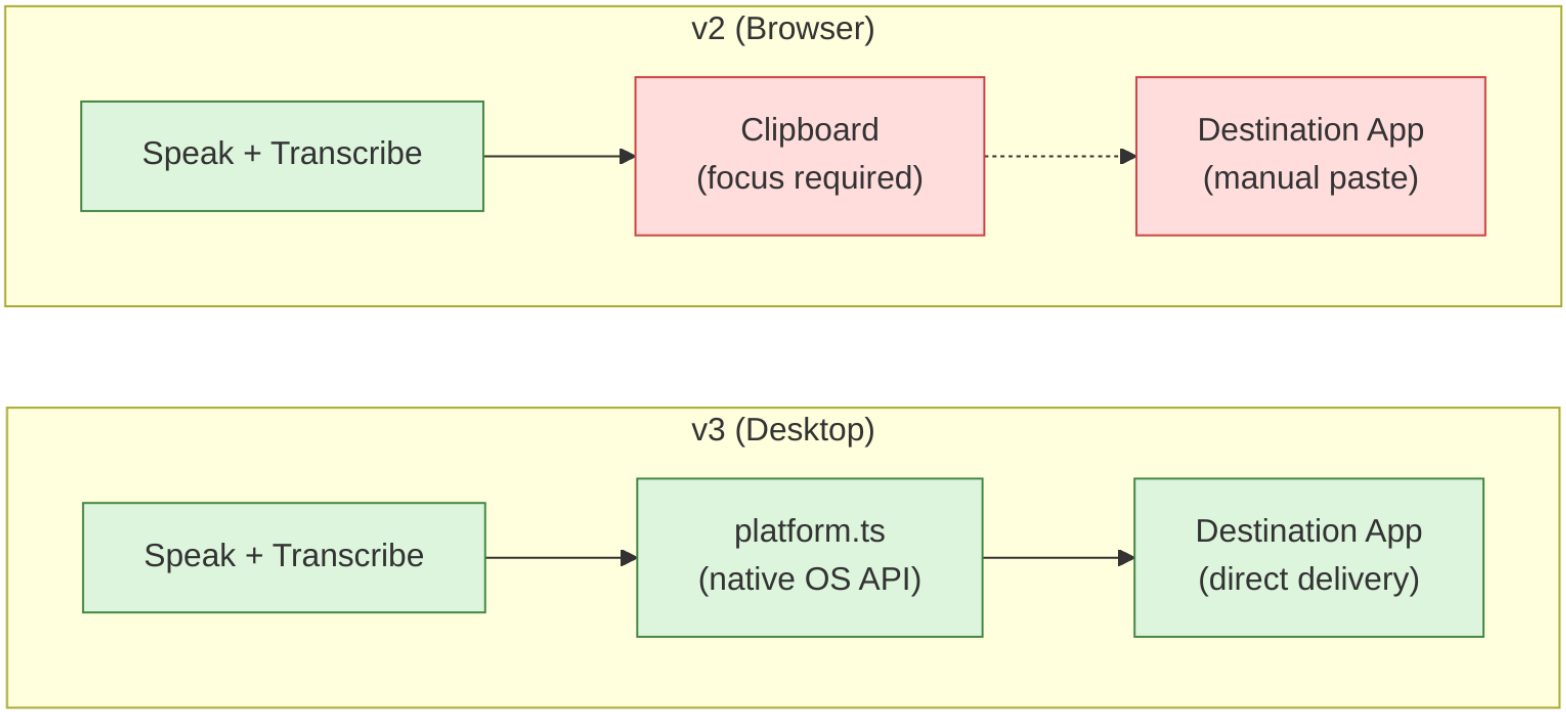
The Clipboard Focus Rule

`navigator.clipboard.writeText()` requires the browser tab to have document focus. If the user has switched to another application — which is the entire point of a transcription tool — the write is blocked.

This is a security boundary enforced by the browser platform. Not a missing feature. Workarounds are fragile and non-portable.

VAD and recording work cross-window — Voice Scribe listens while the user works elsewhere. **Delivery is what breaks.** The full loop requires native OS access.

The Delivery Gap — Diagram



Green = works as intended. Red = browser platform ceiling. v3 closes the last step.

System Architecture at v3

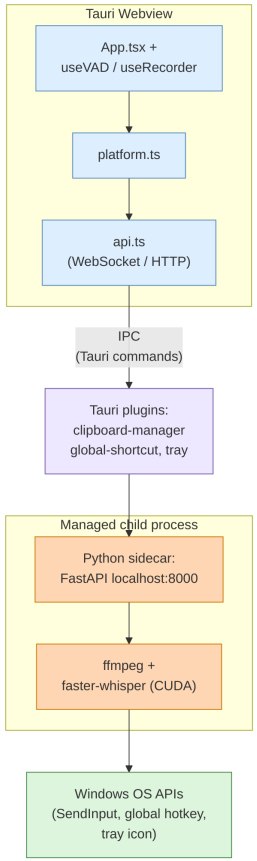
v3 is a platform change, not a rewrite. The same React frontend and Python backend are wrapped in a Tauri desktop application.

- **Tauri** hosts the Vite/React frontend in a native webview — the same build, no component forks
- **Python sidecar** replaces Docker — Tauri spawns `uvicorn app.main:app` as a managed child process
- `platform.ts` routes platform-conditional behavior at the service layer — clipboard, injection, context detection
- **No Docker dependency** at runtime — the model runs bare on the host GPU; Docker path (`make dev`) remains for web development

What is unchanged

All of v2: push-to-talk, continuous mode, VAD, streaming, voice commands (`copy`, `clear`), settings panel, hotkeys. The browser build continues to work without modification.

Architecture — Diagram



Blue = React webview Orange = Python sidecar Lavender = Tauri plugins

Green = OS APIs.

Monorepo Structure

One repository, three app directories. The API is shared between web and desktop.

`apps/web/` — unchanged. Vite frontend. Serves both browser tabs and the Tauri webview.

`apps/api/` — unchanged. FastAPI backend. v2: runs in Docker. v3: runs as a managed sidecar. Same code, different launch context.

`apps/desktop/` — new. Contains `tauri.conf.json` and `src-tauri/` (Rust config and Tauri commands) *only*. No component copies live here.

The Fork Rule

Two copies of a component are two maintenance surfaces that diverge silently.
Capability differences (native clipboard, direct injection) live in `platform.ts` — not in component forks and not in a parallel service file.
If you find yourself copying a component into `apps/desktop/`, stop.

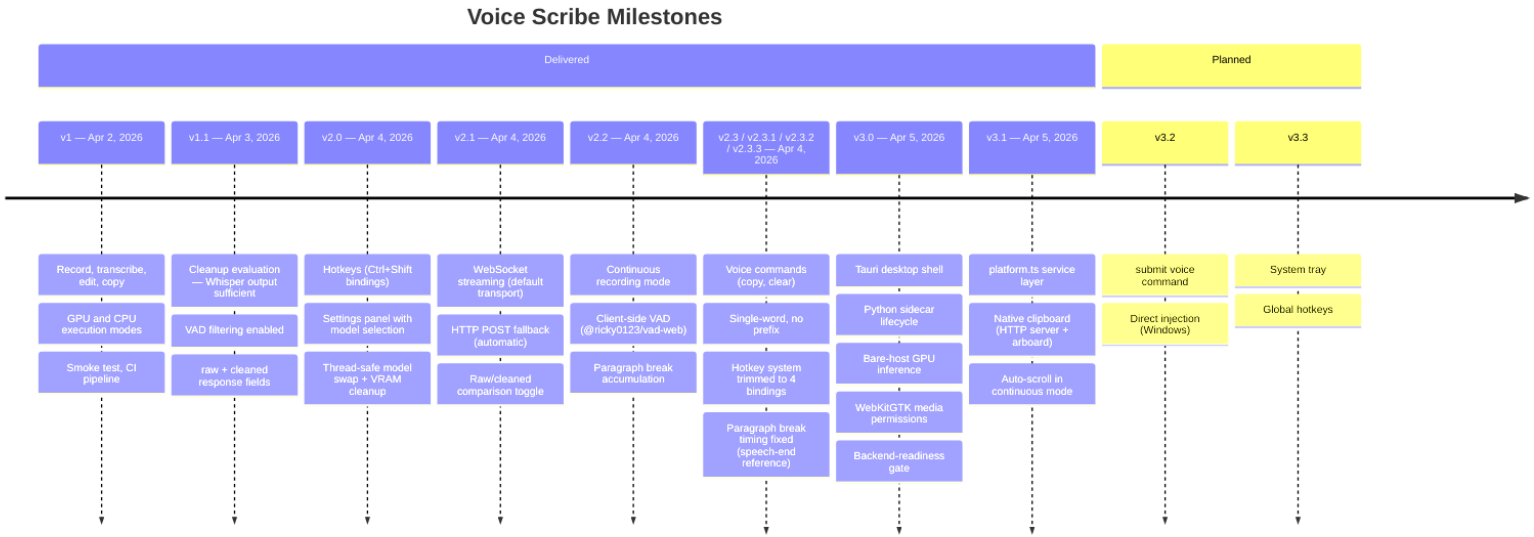
`platform.ts` — The Service Layer

The single routing surface for all platform-conditional behavior. Components call `platform.*` and are unaware of context.

Export	Browser	Tauri (Desktop)
`isDesktop()`	Returns `false`	Returns `true` (`window.__TAURI__`)
`writeClipboard()`	`navigator.clipboard.writeText()`	`fetch()` POST to `:8001` — Rust writes via `arboard`
`submitText()`	Calls `writeClipboard()` (safe fallback)	IPC → Rust → `SendInput` → focused app

Build contract: One Vite build, one artifact. Desktop clipboard uses `fetch()` to a Rust HTTP server — no Tauri plugin imports needed. Browser build produces zero import errors.

Milestone Timeline



Delivered milestones show actual dates and key outputs. Planned milestones show sequence — no hard deadlines.

v3.0 — Tauri Shell + Sidecar Lifecycle

Deliverable: Tauri app launches, manages the Python sidecar, all v2 functionality works identically to the browser version.

Key scope:

- `apps/desktop/` scaffolding: Tauri v2 CLI, `tauri.conf.json`, Cargo project
- Sidecar lifecycle: startup health poll, graceful shutdown (SIGTERM → SIGKILL after timeout), crash surface, port-conflict error dialog
- `config.py` defaults for host: `~/.cache/voice-scribe`, system temp dir, Tauri webview CORS origin
- WebSocket origin enforcement updated; `make desktop` target added

Gate: record → transcribe → display via WebSocket (primary) and HTTP POST (fallback). Close app → no orphan `uvicorn` process. Relaunch works clean.

Not in scope: native clipboard, `submit`, global hotkeys, system tray.

v3.1 — Native Clipboard + Auto-Scroll

Deliverable: `copy` voice command and `Ctrl+Shift+Y` use native OS clipboard. No window focus required.

Key scope:

- `platform.ts` introduced: `isDesktop()`, `writeClipboard()`, `submitText()` stub
- Two `App.tsx` clipboard call sites routed through `platform.writeClipboard()`
- HTTP clipboard server (`tiny_http` + `arboard` on port 8001) — bypasses WebKitGTK user-activation requirement
- `voiceCommands.ts` unchanged; `submit` lands in v3.2
- Continuous mode auto-scroll: pins to bottom, releases on user interaction

Gate: dictate → say "copy" → switch to VS Code → Ctrl+V → transcript is there. Same with Ctrl+Shift+Y. Browser build: zero import errors, all tests pass.

v3.2 — Direct Injection (`submit`)

Deliverable: saying "submit" types the transcript into the focused application. No clipboard. Windows-only for this milestone.

Key scope:

- Rust: `SendInput` via Windows API (`enigo` or equivalent), exposed as a Tauri IPC command
- Hard requirement: restore focus to the previous window before injecting
- `voiceCommands.ts`: `submit` added to `COMMAND_MAP`; `App.tsx` gains `case "submit"` → `platform.submitText()`
- Browser fallback: `submitText()` calls `writeClipboard()`

Gate: focus VS Code, say "submit", transcript appears at cursor — no clipboard. 10 false-positive sentences with "submit" as a non-command word, zero fires. One-week soak period required before marking complete.

Deferred: Linux/macOS injection.

v3.3 — System Tray + Global Hotkeys

Deliverable: Voice Scribe runs in the background. A global hotkey triggers recording from any application.

Key scope:

- System tray: icon, menu (Record, Settings, Quit), status indicator (idle / listening / processing)
- Global hotkeys: `@tauri-apps/plugin-global-shortcut`. Registration failure surfaces as a user-visible error dialog — not a crash. The error message and UX must be implemented and verified in the gate.
- Window management: React state updates while the window is hidden require explicit verification — not assumed

Gate: hide to tray → focus VS Code → press global hotkey → dictate → say "copy" or "submit" → delivered. Window never appeared. Failure path: conflicting hotkey → user-visible error, no crash. Tray Quit: sidecar gone.

Why Python Sidecar, Not Rust Inference?

The decision

Tauri spawns the existing FastAPI backend as a managed child process. No Docker. No rewrite of the inference layer.

Why not rewrite in Rust?

- faster-whisper is proven — accuracy and speed validated in production use
- Building the Tauri shell *and* rewriting inference simultaneously is scope explosion
- The two problems are independent and can be sequenced

When to revisit

Rust-native inference (whisper.cpp) is a **named future milestone**.

Trigger: distribution to non-developer users becomes a requirement.

Single-binary packaging is the forcing function — not performance.

Until then, the Python sidecar is the correct decision, not technical debt.

Why Runtime Detection, Not Build Flags?

The decision: one Vite build serves both the browser and the Tauri webview. Platform context detected at runtime, not build time.

- `window.__TAURI__` — injected by Tauri into the webview. Absent in the browser. `platform.ts` reads this at call time, not at bundle time.
- **No Tauri imports in the clipboard path** — desktop clipboard uses `fetch()` to a Rust HTTP server, not a Tauri plugin. No `@tauri-apps/` imports needed for v3.1.
- **Dynamic `import()` for future plugins** — v3.3 will use `@tauri-apps/plugin-global-shortcut`. Loaded via dynamic `import()`, excluded from `optimizeDeps`.

What this avoids: separate build targets, separate environment variables, build-time divergence. One artifact to test, one to ship.

Why `copy` and `submit` Are Separate Commands

	`copy`	`submit`
Action	Writes to clipboard	Injects keystrokes into focused app
Reversible	Yes	No — undo may be limited
Focus req.	None (native API)	Previous window must be correct
Risk	Overwrites clipboard	May type into wrong application
Browser	`navigator.clipboard`	Falls back to `copy`

Collapsing `copy` and `submit` into one command removes the user's safe fallback. `copy` is predictable. `submit` is irreversible.

The **command separation is non-negotiable**. The **specific words** are subject to gate evaluation before each milestone ships.

Prerequisites Before v3.0

Prereq 1 — `wsCleanupRef` refactor. Standalone PR on `main` before any Tauri code exists.

- Current: single ref, only the last WebSocket cleanup survives. Low-risk in-browser (SPA never navigates).
- v3 risk: Tauri webview persists; "close the panel" is a real unmount. Orphaned connections become real.
- Fix: `Set<() => void>` — register on open, deregister on close, iterate on unmount. Browser-tested before `apps/desktop/` exists.

Prereq 2 — Host environment validation. The API currently runs in Docker. v3 runs it bare on the host.

- Python 3.10+, faster-whisper, ffmpeg on `PATH`, CUDA toolkit
- API starts without Docker; `GET /health` returns `"device": "gpu"`
- Close condition: setup documented in `docs/v3-host-setup.md`

What Is Deferred (Named, Not Forgotten)

- **Model management UI** — download progress, cache management without editing config files. The existing settings panel handles runtime model swap; that is sufficient for v3.0–v3.3. Deferred to v3.x.
- **Linux/macOS `submit` injection** — different APIs (`xdotool`, `CGEvent`), different failure modes. Windows-only is pragmatic scope discipline. Deferred to v3.x.
- **Hotkey configurability** — registration failure is handled and surfaced; the specific keys are not user-configurable in v3.3. Deferred to v3.x.
- **Auto-update** — Tauri has built-in support. Not in scope for any v3 sub-milestone.
- **Rust-native inference** — single-binary distribution. Trigger condition stated above. Named future milestone.